

**LAPORAN PRAKTIKUM SISTEM OPERASI
CPU SCHEDULING**



**Dosen Pengampu :
Triawan Adi Cahyanto M.Kom**

**Disusun Oleh :
Subhan Maulana Andrianto 2410651014**

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025**

1.. EKSPERIMEN CPU SHEDULING

Percobaan menggunakan data 1 :

- First Come First Served

```
root@DESKTOP-J9C60BQ:~# ./fcfs
=== SIMULASI ALGORITMA FCFS ===
Masukkan jumlah proses: 3

Proses P1
  Arrival Time: 0
  Burst Time: 10

Proses P2
  Arrival Time: 1
  Burst Time: 5

Proses P3
  Arrival Time: 2
  Burst Time: 8

=== HASIL PENJADWALAN FCFS ===
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1      0     10    10     10     0
P2      1      5    15     14     9
P3      2      8    23     21    13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33

=== GANTT CHART ===
| P1 | P2 | P3 |
0   10  15  23
```

- Shorted Job First

```
root@DESKTOP-J9C60BQ:~# gcc -o sjf sjf.c
root@DESKTOP-J9C60BQ:~# ./sjf
=== SIMULASI ALGORITMA SJF (Non-Preemptive) ===
Masukkan jumlah proses: 3

Proses P1
  Arrival Time: 0
  Burst Time: 10

Proses P2
  Arrival Time: 1
  Burst Time: 5

Proses P3
  Arrival Time: 2
  Burst Time: 8

=== HASIL PENJADWALAN SJF ===
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1      0     10    10     10     0
P2      1      5    15     14     9
P3      2      8    23     21    13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33

=== URUTAN EKSEKUSI ===
Proses dieksekusi dalam urutan: P1 -> P2 -> P3
```

Percobaan menggunakan data 2 :

- First Come First Served

```
root@DESKTOP-J9C60BQ:~# gcc -o fcfs fcfs.c
root@DESKTOP-J9C60BQ:~# ./fcfs
=== SIMULASI ALGORITMA FCFS ===
Masukkan jumlah proses: 4

Proses P1
Arrival Time: 0
Burst Time: 5

Proses P2
Arrival Time: 1
Burst Time: 3

Proses P3
Arrival Time: 2
Burst Time: 8

Proses P4
Arrival Time: 3
Burst Time: 6

=== HASIL PENJADWALAN FCFS ===
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1      0      5      5      5      0
P2      1      3      8      7      4
P3      2      8     16     14      6
P4      3      6     22     19     13

Average Turnaround Time: 11.25
Average Waiting Time: 5.75

=== GANTT CHART ===
| P1 | P2 | P3 | P4 |
0   5   8   16  22
```

- Shorted Job First

```
root@DESKTOP-J9C60BQ:~# gcc -o sjf sjf.c
root@DESKTOP-J9C60BQ:~# ./sjf
=== SIMULASI ALGORITMA SJF (Non-Preemptive) ===
Masukkan jumlah proses: 4

Proses P1
Arrival Time: 0
Burst Time: 5

Proses P2
Arrival Time: 1
Burst Time: 3

Proses P3
Arrival Time: 2
Burst Time: 8

Proses P4
Arrival Time: 3
Burst Time: 6

=== HASIL PENJADWALAN SJF ===
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1      0      5      5      5      0
P2      1      3      8      7      4
P3      2      8     22     20     12
P4      3      6     14     11      5

Average Turnaround Time: 10.75
Average Waiting Time: 5.25

=== URUTAN EKSEKUSI ===
Proses dieksekusi dalam urutan: P1 -> P2 -> P4 -> P3
```

2. Analisis Eksperimen ketiga algoritma

Jawab pertanyaan berikut:

1. Algoritma mana yang memberikan Average Waiting Time terkecil?
2. Mengapa hasil algoritma berbeda-beda?
3. Kapan sebaiknya menggunakan algoritma FCFS?
4. Kapan sebaiknya menggunakan algoritma SJF?
5. Kapan sebaiknya menggunakan algoritma Round Robin?

Jawaban :

1. Algoritma SJF (Shortest Job First) memberikan *Average Waiting Time* terkecil.

Baik pada Data Set 1 maupun Data Set 2, hasil rata-rata waktu tunggu selalu paling kecil dibanding FCFS dan Round Robin.

2. Karena setiap algoritma memiliki kebijakan penjadwalan yang berbeda dalam menentukan urutan eksekusi proses.

3. Gunakan FCFS (First Come First Served) ketika:

- Sistem memiliki beban proses besar dan datang berurutan, seperti *batch processing* atau *job queue*.
- Tidak masalah jika ada perbedaan besar antara waktu eksekusi proses.
- Tujuannya adalah kesederhanaan dan fairness berdasarkan urutan kedatangan.

4. Gunakan SJF (Shortest Job First) ketika:

- Sistem mengetahui atau dapat memperkirakan burst time (durasi eksekusi) setiap proses.
- Tujuannya adalah efisiensi maksimum — meminimalkan *waiting time* dan *turnaround time*.
- Proses kecil sering datang lebih banyak daripada proses besar.

5. Gunakan Round Robin (RR) ketika:

- Sistem bersifat interaktif atau multitasking, di mana setiap proses harus mendapatkan waktu CPU secara bergantian.
- Tujuannya adalah keadilan (fairness) antar proses dan respons cepat terhadap input pengguna.

3. Modifikasi Program

A. Tugas Tambahkan fitur berikut pada program FCFS:

1. Tampilkan Gantt Chart yang lebih detail
2. Simpan hasil ke file text
3. Baca input dari file

B. Langkah Langkah Tambahkan 3 method pada program fcfs yang kita sudah buat

```
GNU nano 7.2                                fcfs_mod.c
#include <stdio.h>
#include <stdlib.h>

struct Process {
    int pid;
    int arrival_time;
    int burst_time;
    int waiting_time;
    int turnaround_time;
    int completion_time;
};

// ===== FUNGSI UNTUK MEMBACA INPUT DARI FILE =====
int read_input_from_file(const char *filename, struct Process proc[]) {
    FILE *f = fopen(filename, "r");
    if (f == NULL) {
        printf("Gagal membuka file input: %s\n", filename);
        exit(1);
    }

    int n;
    fscanf(f, "%d", &n); // baris pertama = jumlah proses
    for (int i = 0; i < n; i++) {
        fscanf(f, "%d %d", &proc[i].arrival_time, &proc[i].burst_time);
        proc[i].pid = i + 1;
    }
    fclose(f);
    return n;
}

// ===== FUNGSI UNTUK MENAMPILKAN GANTT CHART =====
void display_gantt_chart(struct Process proc[], int n) {
    printf("\n=== GANTT CHART ===\n");

    // Baris proses
    printf("|");
    for (int i = 0; i < n; i++) {
        printf(" P%d |", proc[i].pid);
    }
    printf("\n");

    // Baris waktu
    printf("%d", 0);
    for (int i = 0; i < n; i++) {
        printf(" %d", proc[i].completion_time);
    }
    printf("\n");
}

// ===== FUNGSI UNTUK MENYIMPAN HASIL KE FILE =====
void save_to_file(const char *filename, struct Process proc[], int n, float avg_tat, float avg_wt) {
    FILE *f = fopen(filename, "w");
    if (f == NULL) {
        printf("Gagal menyimpan file hasil!\n");
        return;
    }

    fprintf(f, "=== HASIL PENJADWALAN FCFS ===\n");
    fprintf(f, "PID\tAT\tBT\tCT\tWT\n");
    for (int i = 0; i < n; i++) {
        fprintf(f, "P%d\t%d\t%d\t%d\t%d\n",
            proc[i].pid,
            proc[i].arrival_time,
            proc[i].burst_time,
            proc[i].completion_time,
            proc[i].turnaround_time,
            proc[i].waiting_time);
    }

    fprintf(f, "\nAverage Turnaround Time: %.2f\n", avg_tat);
    fprintf(f, "Average Waiting Time: %.2f\n", avg_wt);
}
```

```

fclose(f);
printf("\nHasil berhasil disimpan ke file: %s\n", filename);
}

// ===== PROGRAM UTAMA =====
int main() {
    struct Process proc[100];
    int n;

    printf("=== SIMULASI ALGORITMA FCFS (Modifikasi) ===\n");
    printf("Baca input dari file 'input.txt'...\n");

    // Baca data dari file input
    n = read_input_from_file("input.txt", proc);

    // Hitung waktu komputasi
    int current_time = 0;
    for (int i = 0; i < n; i++) {
        if (current_time < proc[i].arrival_time)
            current_time = proc[i].arrival_time;

        proc[i].completion_time = current_time + proc[i].burst_time;
        proc[i].turnaround_time = proc[i].completion_time - proc[i].arrival_time;
        proc[i].waiting_time = proc[i].turnaround_time - proc[i].burst_time;
        current_time = proc[i].completion_time;
    }

    // Tampilkan hasil di layar
    printf("\n=== HASIL PENJADWALAN FCFS ===\n");
    printf("PID\tAT\tBT\tCT\tTAT\tWT\n");

    float total_tat = 0, total_wt = 0;
    for (int i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
            proc[i].pid,
            proc[i].arrival_time,
            proc[i].burst_time,
            proc[i].completion_time,
            proc[i].turnaround_time,
            proc[i].waiting_time);
        total_tat += proc[i].turnaround_time;
        total_wt += proc[i].waiting_time;
    }

    float avg_tat = total_tat / n;
    float avg_wt = total_wt / n;

    printf("\nAverage Turnaround Time: %.2f\n", avg_tat);
    printf("Average Waiting Time: %.2f\n", avg_wt);

    // Tampilkan Gantt Chart
    display_gantt_chart(proc, n);

    // Simpan ke file hasil
    save_to_file("hasil_fcfs.txt", proc, n, avg_tat, avg_wt);

    return 0;
}

```

Lalu compile, setelah di compile masukan buat file incex.txt Dan masukan text seperti format di bawah ini

```

GNU nano 7.2                                     input.txt
3
0 10
1 5
2 8

```

Jika dirun maka akan keluar output seperti dibawah ini :

```

root@DESKTOP-J9C60BQ:/# gcc -o fcfs_mod fcfs_mod.c
root@DESKTOP-J9C60BQ:/# ./fcfs_mod
=== SIMULASI ALGORITMA FCFS (Modifikasi) ===
Baca input dari file 'input.txt'...

=== HASIL PENJADWALAN FCFS ===
PID    AT     BT     CT     TAT     WT
P1      0      10     10     10       0
P2      1       5     15     14       9
P3      2       8     23     21     13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33

=== GANTT CHART ===
| P1 | P2 | P3 |
0   10  15  23

```

4. Eksperimen dengan data berbeda untuk perbandingan algoritma

A. Proses dengan burst time bervariasi

```
root@DESKTOP-J9C60BQ:~# ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 4
Masukkan time quantum: 3

Proses P1
  Arrival Time: 0
  Burst Time: 24

Proses P2
  Arrival Time: 1
  Burst Time: 3

Proses P3
  Arrival Time: 2
  Burst Time: 3

Proses P4
  Arrival Time: 3
  Burst Time: 6

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Sisa waktu: 21
Waktu 3: Menjalankan P1 -> Sisa waktu: 18
Waktu 6: Menjalankan P2 -> Selesai pada waktu 9
Waktu 9: Menjalankan P3 -> Selesai pada waktu 12
Waktu 12: Menjalankan P4 -> Sisa waktu: 3
Waktu 15: Menjalankan P1 -> Sisa waktu: 15
Waktu 18: Menjalankan P1 -> Sisa waktu: 12
Waktu 21: Menjalankan P1 -> Sisa waktu: 9
Waktu 24: Menjalankan P4 -> Selesai pada waktu 27
Waktu 27: Menjalankan P4 -> Selesai pada waktu 27

=== HASIL PENJADWALAN ROUND ROBIN ===


| PID | AT | BT | CT        | TAT   | WT          |  |
|-----|----|----|-----------|-------|-------------|--|
| P1  | 0  | 24 | 130579930 | 32767 | -1726173360 |  |
| P2  | 1  | 3  | 9         | 8     | 5           |  |
| P3  | 2  | 3  | 12        | 10    | 7           |  |
| P4  | 3  | 6  | 27        | 24    | 18          |  |


Average Turnaround Time: 8202.25
Average Waiting Time: -431543328.00
```

B. Proses datang bersamaan

```
root@DESKTOP-J9C60BQ:~# ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 5
Masukkan time quantum: 4

Proses P1
  Arrival Time: 0
  Burst Time: 1

Proses P2
  Arrival Time: 0
  Burst Time: 2

Proses P3
  Arrival Time: 0
  Burst Time: 3

Proses P4
  Arrival Time: 0
  Burst Time: 4

Proses P5
  Arrival Time: 0
  Burst Time: 5

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Selesai pada waktu 1
Waktu 1: Menjalankan P2 -> Selesai pada waktu 3
Waktu 3: Menjalankan P3 -> Selesai pada waktu 6
Waktu 6: Menjalankan P4 -> Selesai pada waktu 10
Waktu 10: Menjalankan P5 -> Sisa waktu: 1
Waktu 14: Menjalankan P5 -> Selesai pada waktu 15

=== HASIL PENJADWALAN ROUND ROBIN ===


| PID | AT | BT | CT | TAT | WT |
|-----|----|----|----|-----|----|
| P1  | 0  | 1  | 1  | 1   | 0  |
| P2  | 0  | 2  | 3  | 3   | 1  |
| P3  | 0  | 3  | 6  | 6   | 3  |
| P4  | 0  | 4  | 10 | 10  | 6  |
| P5  | 0  | 5  | 15 | 15  | 10 |


Average Turnaround Time: 7.00
Average Waiting Time: 4.00
```


C. Proses datang dengan interval

```
root@DESKTOP-J9C60BQ:~# ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 5
Masukkan time quantum: 3

Proses P1
  Arrival Time: 0
  Burst Time: 2

Proses P2
  Arrival Time: 4
  Burst Time: 6

Proses P3
  Arrival Time: 5
  Burst Time: 10

Proses P4
  Arrival Time: 7
  Burst Time: 4

Proses P5
  Arrival Time: 9
  Burst Time: 3

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Selesai pada waktu 2
Waktu 4: Menjalankan P2 -> Sisa waktu: 3
Waktu 7: Menjalankan P2 -> Selesai pada waktu 10
Waktu 10: Menjalankan P3 -> Sisa waktu: 7
Waktu 13: Menjalankan P4 -> Sisa waktu: 1
Waktu 16: Menjalankan P2 -> Selesai pada waktu 16
Waktu 16: Menjalankan P5 -> Selesai pada waktu 19
Waktu 19: Menjalankan P3 -> Sisa waktu: 4
Waktu 22: Menjalankan P3 -> Sisa waktu: 1
Waktu 25: Menjalankan P4 -> Selesai pada waktu 26

=== HASIL PENJADWALAN ROUND ROBIN ===
PID      AT      BT      CT      TAT      WT
----      --      --      --      ---      --
P1        0        2        2        2        0
P2        4        6       16       12        6
P3        5       10    -2067554209  29349    489341664
P4        7        4       26       19       15
P5        9        3       19       10        7

Average Turnaround Time: 5878.40
Average Waiting Time: 97868336.00
```

5. Analisis Eksperimen perbandingan algoritma

1. Analisis Performa

FCFS (First-Come, First-Served) memberikan hasil terbaik (Average Waiting Time / AWT terkecil, mendekati algoritma lain) ketika:

- Semua proses memiliki Burst Time (BT) yang hampir sama. Ketika semua proses memiliki BT yang seragam, urutan eksekusi tidak terlalu memengaruhi waktu tunggu secara keseluruhan.
- Proses dengan BT kecil datang lebih dulu. Jika proses-proses pendek datang pertama, FCFS akan menyelesaikannya dengan cepat, menghasilkan AWT yang rendah.
- Tidak ada Convoy Effect. FCFS paling menderita oleh Convoy Effect, di mana proses dengan BT yang sangat besar (seperti P1: BT=24 di Test Case 1) datang di awal, memaksa semua proses kecil menunggu lama. FCFS akan memberikan hasil terbaik ketika Convoy Effect ini tidak terjadi.

Mengapa SJF hampir selalu memberikan Average WT terkecil?

SJF (Shortest Job First) hampir selalu memberikan Average Waiting Time (AWT) terkecil karena SJF adalah algoritma optimal dalam hal ini.

- Strategi dasarnya adalah menyelesaikan proses yang membutuhkan waktu CPU paling sedikit terlebih dahulu.
- Dengan segera mengeluarkan proses-proses pendek dari antrean, SJF meminimalkan waktu tunggu yang harus ditanggung oleh sejumlah besar proses, sehingga AWT keseluruhan menjadi yang terendah.

2. Context Switching

Algoritma mana yang paling banyak melakukan context switch?

Algoritma Round Robin (RR) adalah yang paling banyak melakukan context switch

- Setiap kali jatah waktu (time quantum) habis atau proses selesai, sistem harus melakukan context switch untuk beralih ke proses berikutnya di antrean Ready.

Bagaimana pengaruh quantum terhadap jumlah context switch di Round Robin?

- Quantum Kecil: $\downarrow$ Jumlah Context Switch $\uparrow$ (Tinggi). Jika quantum sangat kecil (misalnya 1), setiap proses akan berganti state setelah setiap unit waktu, menghasilkan overhead yang sangat besar.
- Quantum Besar: $\downarrow$ Jumlah Context Switch $\downarrow$ (Rendah). Jika quantum sangat besar, RR akan berperilaku mirip dengan FCFS, karena proses yang baru masuk mungkin akan selesai sebelum quantum-nya habis.

Coba jalankan Round Robin dengan quantum 2, 4, 8, 16 - apa yang terjadi?

- Quantum Kecil (2, 4): Memberikan respon yang sangat cepat (Response Time) karena proses segera mendapatkan CPU. Namun, AWT dan overhead context switch akan tinggi. Untuk Test Case 3 (Quantum=2), context switch sangat intens, namun

memastikan semua proses (terutama P2, P4, P5 yang relatif pendek) mendapat jatah eksekusi cepat.

- Quantum Sedang (8): Menghasilkan trade-off yang seimbang. Overhead context switch berkurang, sementara Response Time masih cukup baik. Ini sering menjadi pilihan optimal.
- Quantum Besar (16): Perilaku RR akan mendekati FCFS. Jika quantum lebih besar dari sebagian besar Burst Time, proses-proses tersebut akan selesai dalam satu slice waktu, sehingga context switch hanya terjadi ketika proses selesai, mirip FCFS.

3. Fairness

Algoritma mana yang paling "adil" untuk semua proses?

Algoritma Round Robin (RR) adalah yang paling "adil" (Fair).

- Keadilan di sini didefinisikan sebagai perlakuan yang merata atau pembagian waktu CPU yang sama untuk semua proses, tanpa memandang Arrival Time atau Burst Time.
- RR menjamin bahwa tidak ada proses yang akan "kelaparan" dan setiap proses akan mendapat kesempatan eksekusi dalam waktu yang singkat.

Proses mana yang paling dirugikan di algoritma FCFS?

Proses yang paling dirugikan di algoritma FCFS adalah proses dengan Burst Time (BT) kecil yang datang setelah proses dengan BT yang sangat besar.

- Contohnya di Test Case 1, P2, P3, dan P4 dirugikan karena harus menunggu P1 dengan BT=24 selesai terlebih dahulu (meskipun P1 datang paling awal). Proses P2 (BT=3) harus menunggu hingga waktu 24 (CT P1) baru bisa mulai, padahal ia datang di waktu 1.

Apakah ada kemungkinan starvation di SJF?

Ya, ada kemungkinan starvation (kelaparan) di SJF.

- Starvation terjadi ketika proses dengan Burst Time yang sangat panjang terus-menerus terhalang (tidak pernah mendapat CPU) karena selalu ada proses baru dengan Burst Time yang lebih pendek (atau sisa waktu yang lebih pendek) yang tiba di sistem.
- Hal ini sangat mungkin terjadi di sistem dengan beban kerja yang tinggi (banyak proses pendek baru terus datang).
- Solusi untuk mencegah starvation di SJF adalah dengan teknik Aging, yaitu meningkatkan prioritas suatu proses seiring bertambahnya waktu tunggu.

4. Real World Application

Untuk web server yang melayani banyak request kecil, algoritma mana yang cocok?

Round Robin (RR).

- RR memastikan Response Time yang cepat dan keadilan (semua request dilayani secara bergiliran), memberikan pengalaman pengguna yang baik.

Untuk render video (task besar), algoritma mana yang cocok?

Shortest Job First (SJF) atau Prioritas Non-Preemptive.

- Jika kita tahu durasi rendering (BT) dan ingin menyelesaikan render tercepat terlebih dahulu, SJF optimal untuk throughput.
- Namun, untuk tugas yang benar-benar besar seperti video rendering, Prioritas Non-Preemptive (di mana tugas rendering diberikan prioritas rendah namun setelah dimulai tidak dapat disela) atau FCFS bisa digunakan jika rendering tersebut adalah satu-satunya tugas yang penting. SJF yang paling logis jika tujuannya adalah meminimalkan waktu penyelesaian tugas pendek-menengah.

Untuk OS desktop (interactive), algoritma mana yang cocok?

Round Robin (RR) dengan Skema Prioritas Dinamis.

- OS desktop (Windows, macOS, Linux) adalah sistem time-sharing yang sangat interaktif.
- RR digunakan sebagai dasar untuk memberikan Response Time yang cepat (misalnya, saat klik mouse atau mengetik).
- Algoritma modern menggabungkan RR dengan Skema Prioritas (prioritas tinggi untuk tugas foreground/interactive dan prioritas rendah untuk tugas background) dan Prioritas Dinamis (prioritas berubah berdasarkan perilaku proses) untuk memberikan fairness dan responsivitas terbaik.